End of Day Report

Name	Atibali Saiyed
Date	03 November 2026
Repository	https://github.com/Atibali/Gateway-Training/tree/main/Week1/Day1

Part 1 — C# Fundamentals

.NET basics — Learned what .NET is (a cross-platform runtime and SDK), how to create a project with `dotnet new console`, build with `dotnet build`, and run with `dotnet run`. Understood the project structure (`.csproj`, `Program.cs`, `bin/`, `obj/`) and how top-level statements let you write code without an explicit `Main` method.

Types and variables — Learned the difference between value types (`int`, `long`, `double`, `decimal`, `bool`, `char`) and reference types (`string`, `object`, arrays). Saw that value types are copied on assignment while reference types share the same object. Used `var` for type inference, `const` for constants, and nullable types (`int?`) for values that can be null.

Type conversion — Learned implicit casting (safe widening) versus explicit casting (narrowing, which may lose data), plus `Parse` (throws on failure), `TryParse` (safe, returns a Boolean), and `Convert` (handles null). Also used pattern matching (`is`, switch type patterns) for safe downcasts.

Operators — Practised arithmetic (`+ - * / %`), comparison (`== != < > <= >=`), logical (`&& || !`), ternary (`?:`), null-coalescing (`??`), and null-conditional (`?.`) operators.

Control flow — Used `if/else`, `switch` statements, switch expressions (with relational patterns such as `< 0` and `or` patterns such as `1 or 2 or 3`), `for`, `foreach`, `while`, `do-while`, `break`, and `continue`.

Methods — Learned parameters and return values, method overloading (same name, different signatures), optional and named parameters, `ref` (modify the caller's variable), `out` (return extra values), `in` (read-only reference), and expression-bodied methods (`=>`).

Strings — Used interpolation (`$"..."`), alignment (`{age,5}`), format specifiers (`{Math.PI:F2}`), and common methods: `Trim`, `ToUpper`, `Replace`, `Substring`, `Length`, `Contains`, `StartsWith`, `EndsWith`, `IndexOf`, and `Split`. Understood that strings are immutable—each operation returns a new string—and used `StringBuilder` for efficient concatenation.

Collections — Worked with arrays (fixed size), `List<T>` (dynamic), `Dictionary<TKey,TValue>` (hash map, with `TryGetValue` for safe access), and `HashSet<T>` (unique elements, with `UnionWith`, `IntersectWith`, and `ExceptWith`). Looped over each with `foreach` and used LINQ (`Where`, `OrderBy`, `ToList`).

Part 1 — Git and Version Control

Version control concepts — Learned what a repository is, what a commit captures (snapshot, message, author, and parent), how history forms a directed acyclic graph (DAG), and why teams use Git for collaboration, audit trails, branching, rollback, and review.

Everyday commands — Practised `init`, `clone`, `status`, `add`, `commit`, `log`, and `diff`, and configured `.gitignore` to exclude `bin/`, `obj/`, and `*.user`.

Branches — Learned to create branches (`git branch`), switch branches (`git switch` / `git checkout`), and merge (`git merge`), plus how merge conflicts occur and how to resolve them.

Remotes — Used `git remote -v`, `git push`, `git pull` (fetch followed by merge), and `git fetch` (download changes without merging).

Pull requests — Understood the workflow: open a pull request (PR) from a feature branch, receive comments and approval from reviewers, then squash-merge to keep history linear.

Good habits — Practised keeping commits small, writing clear imperative commit messages, and pulling before pushing.

Part 2 — QA Manual Testing

Why testing matters — Learned what quality means (fitness for purpose and meeting user expectations), that finding bugs late can cost 10–100 times more, and the chain error → defect → failure: a human error introduces a defect, and the defect causes a failure when executed. Also learned that root cause is the underlying reason, not the visible symptom.

The seven testing principles — Testing shows that defects are present, not that they are absent; exhaustive testing is impossible; test early (shift-left); defects cluster together (Pareto principle); the pesticide paradox means repeating the same tests eventually stops finding new bugs; testing depends on context; and the absence-of-errors fallacy means a bug-free product can still fail if it does not meet user needs.

SDLC models — Covered Waterfall (testing is a late phase), the V-Model (each development phase has a corresponding test phase), and Agile/Scrum (testing is continuous across sprints). Learned where testing fits in each model.

STLC phases — Requirement analysis → Test planning → Test design → Test environment setup → Test execution → Test closure.

Test levels — Unit testing (component), Integration testing, System testing, and Acceptance testing (UAT, alpha, and beta).

Test types — Functional versus non-functional (performance, security, usability, accessibility, and compatibility); black-box versus white-box; retesting (verify a fix) versus regression testing (check for side effects); and smoke testing (broad, shallow) versus sanity testing (narrow, deep).

Static testing — Reviewed requirements and user stories before code exists. Review types covered: informal review, walkthrough, technical review, and inspection (the most formal).

Quality characteristics (ISO/IEC 25010) — Covered functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

Summary

Today I learned the fundamentals of C# (types, conversions, operators, control flow, methods, strings, and collections), how .NET projects are structured and run, the everyday Git workflow from repository initialization to pull requests, and the core concepts of manual testing—including principles, SDLC/STLC, test levels and types, static testing, and ISO/IEC 25010 quality characteristics.

Practical progress: All eight C# lesson files are written and runnable. The code will be pushed to the GitHub repository listed above.